



Final presentation: Segfault Slayer

Schmölz Lukas, Schöpf Emanuel, Schweigl Dominik

Used Tools

Build Tools

- SFML
- Tiled + Tiled
- Catch2

Asset Sources

- Pixel art: Pixellab, Gemini
- Audio: Pixabay (Sounds), itch.io (Music)

Game Overview

Game Idea

- Metroidvania-style 2D side-scroller
- Player fights programming concepts and bugs
- Playful, Pixel-theme

World Design

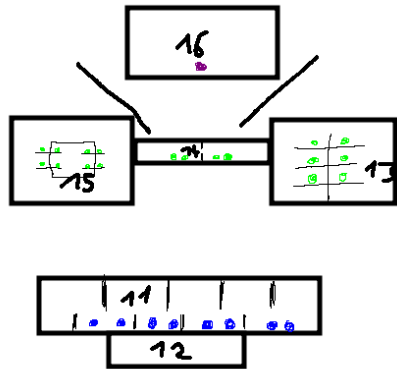
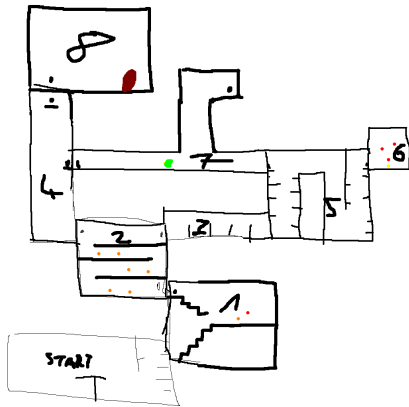
- First Area: Hardware with electric components roaming around
- Second Area: Abstract programming concepts escaped from the bosses lab

Team Responsibilities

- Emanuel
 - (1) map
 - (1) save / load with dedicated save points (rooms)
 - (2) two consecutive areas
- Lukas
 - (2) enemies
 - (1) menus
- Dominik
 - (2) basic movement
 - (1) one or more advanced movement mechanics
 - (1) main combat

Minimaps

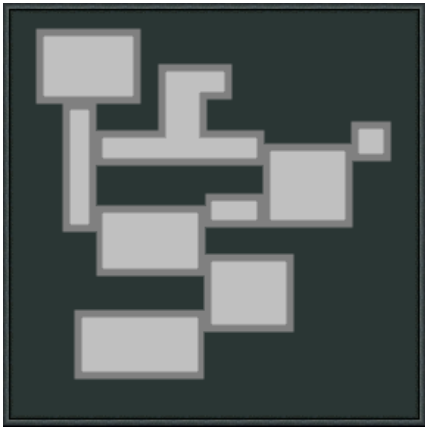
Initial Scribbles:



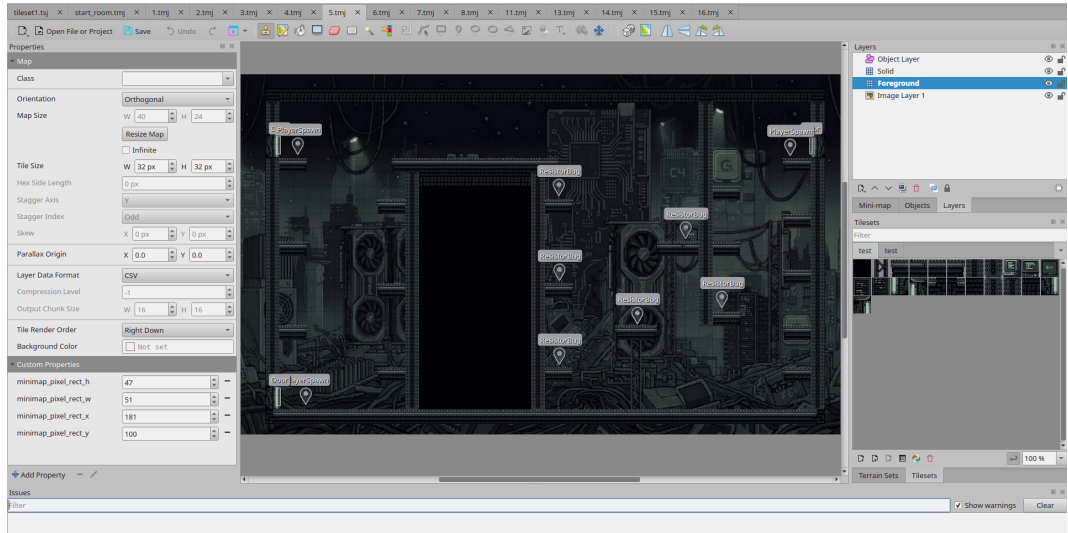
Minimaps

current:

(work in progress)



Our Tiled Setup



Data Design

Flexibility & Extensibility

some examples:

- Loot Drops
- Door Locked
- Room Minimap FloatRect

minimap_pixel_rect_h	37	▲▼	—
minimap_pixel_rect_w	65	▲▼	—
minimap_pixel_rect_x	67	▲▼	—
minimap_pixel_rect_y	143	▲▼	—

drop_chance	0.4	▲▼	—
▼ drop_items	2 items	+ Add	▼ —
[0]	HealPotionItem	...	—
[1]	JumpPotionItem	...	—

Saving and Loading World

```
json j;
try {
    j["player"] = player.serialize();

    j["rooms"] = json::array();
    for (const auto &room : rooms) {
        json j_room = room.second.serialize();
        j_room["id"] = room.first;
        j["rooms"].push_back(j_room);
    }

    j["currentRoom"] = getCurrentRoomId();

    std::ofstream file("saves/" + worldName + ".json");
    if (!file.is_open()) {
        std::cerr << "Failed to open save file\n";
        return;
    }

    file << j.dump(4);
} catch (const std::exception &e) {
    std::cerr << "Serialization error: " << e.what() << "\n";
}
```

```
try {
    if (j.contains("player")) {
        player.deserialize(j["player"]);
    }

    if (j.contains("rooms")) {
        for (const auto &j_room : j["rooms"]) {
            std::string id = "";
            if (j_room.contains("id"))
                id = j_room["id"];

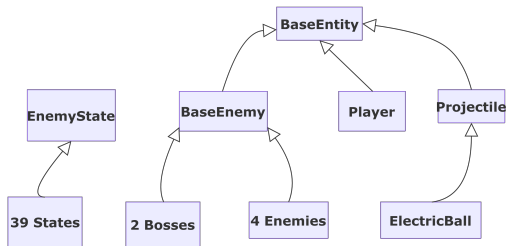
            if (rooms.find(id) == rooms.end()) {
                std::cerr << "Unknown room id in save: " << id << "\n";
                continue;
            }

            Room &room = rooms[id];
            room.deserialize(j_room);
        }
    }

    if (j.contains("currentRoom")) {
        setCurrentRoom(j["currentRoom"]);
    }
} catch (const std::exception &e) {
    std::cerr << "World loading error: " << e.what() << "\n";
}
```

Serialization

Peter Thoman: Handle serialization continuously instead of implementing everything at once at the end.



```
class BaseEntity {  
public:  
    virtual json serialize() const;  
    virtual void deserialize(const json &j);  
};
```

Factory Pattern

```
using json = nlohmann::json;

struct EnemyFactory {
    static std::unique_ptr<BaseEnemy> create(const json &j)
    {
        if (j.empty())
            return nullptr;
        const std::string type = j["type"];

        if (type == "RaceConditionSlime") {
            auto enemy = std::make_unique<RaceConditionSlime>(sf::Vector2f{0.f, 0.f}, BaseEnemy::DROP_CHANCE);
            enemy->deserialize(j);
            return enemy;
        }

        if (type == "Capacitor") {
            auto enemy = std::make_unique<Capacitor>(sf::Vector2f{0.f, 0.f}, BaseEnemy::DROP_CHANCE);
            enemy->deserialize(j);
            return enemy;
        }
    }
};
```

Demo & Code Review

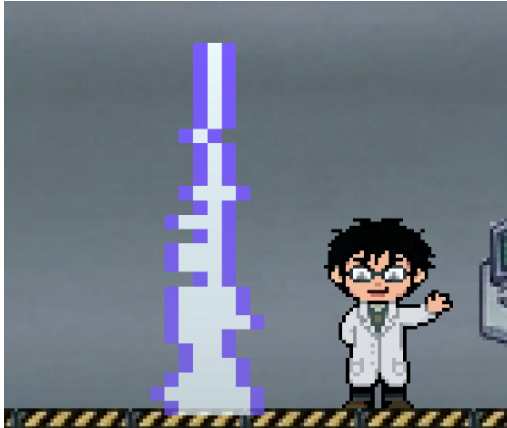
Boss Design

Transistor

- Roams the boss room and shoots projectiles
- Shocks area around him when player too close
- Spawns additional minions in phase 2



Boss Design

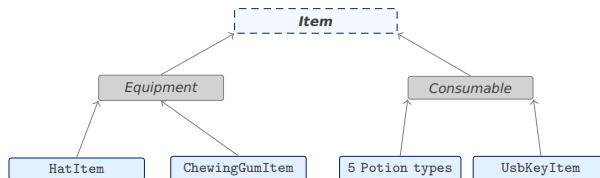


Segfault

- Spawns area attacks and environment blockers
- Summons minions in phase 2
- Glitches into 2 versions of himself in phase 3

Items

- **Equipment** – wearable items occupying a fixed slot; `activate()` moves to `equipmentSlot()` returning the target `SlotKind`
- **Consumable** – single-use items; `activate()` applies the effect and removes the item from inventory



Equipment



Hat



Gum



Backup

Consumables



Heal



Speed



Jump



Damage

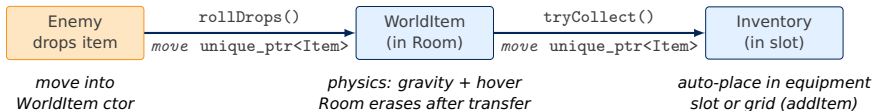


Resist



USB Key

WorldItem & Inventory: Ownership Chain



Transfer 1 – drop (game_scene.cpp)

```
for (auto& enemy : room->enemies_) {
    if (!enemy->isAlive()) {
        for (auto& drop : enemy->rollDrops()) {
            auto worldItem = make_unique<WorldItem>(
                enemy->getPosition(),
                std::move(drop)
            );
            room->appendItem(worldItem);
        }
    }
}
```

Transfer 2 – collection (game_scene.cpp)

```
auto collected = worldItem->tryCollect(
    player_.getBounds()
);
if (collected)
    auto inv = player_.inventory()
    inv.addItem(
        std::move(collected)
    );

// Room:
items_.erase(
    remove_if(items_.begin(), items_.end(),
        [](const auto& i){return i->isCollected();}
    ),
    items_.end()
);
```


Inventory System

Uniform Slot Addressing

- Every slot is a single SlotRef
- moveItem(from, to) and interact(slot, player) are generic entry points

```
enum class SlotKind {  
    Hat, Gum, Backup, Grid, Hotbar  
};  
struct SlotRef { SlotKind kind; int index; };
```

Slot-Move Validation

- Validation lives in the inventory model
- Each item knows which slot kinds it can occupy
- Inventory checks isValidInSlot(item, slot)
- UI drag-drop and keyboard shortcuts obeys these rules

```
bool isValidInSlot(const Item& item, SlotRef s) {  
    const auto equipment = item.equipmentSlot();  
    for (const SlotRef &s : EQUIPMENT_SLOTS_) {  
        if (s.kind == slotKind)  
            return equipment.has_value() &&  
                *equipment == slotKind;  
    }  
    return true;  
}
```

Item Self-Activation

- Inventory calls item.activate(...)
- Logic lives in the item
- Inventory is a passive store

```
// Equipment (e.g. HatItem)  
void activate(...) {  
    inventory.moveToEquipmentSlot(ownSlot);  
}  
// Consumable (e.g. HealingPotionItem)  
void activate(...) {  
    player.heal(amount);  
    inventory.clearSlot(ownSlot);  
}
```

The inventory calls item.activate(player, *this, slot). The item decides what to do – the inventory is a passive store; **logic lives in the item.**

Final Sprint

- Last Music and Audio wiring
- Second Area rework
- Story snippets
- Refactoring



Lessons Learned

- Focus on creating a MVP early
- Creative Work just as important as gameplay mechanics
- Better Milestone / Issue Management